

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour
Total Marks:50

Design Task

Code of Conduct:

I G Amrutha(192511250) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

amrutha

Design Task

1. **Hybrid AI Recommendation Engine Design**
Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. **Smart Disaster Detection System**
Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework**
Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. **Autonomous Supply Chain Optimization System**
Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. **Personalized Learning Analytics Platform**
Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed

Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

Recommended topic: Hybrid AI Recommendation Engine

You can structure your submission like this:

1. Title

- Hybrid AI-Based Recommendation Engine Using PySpark RDDs and Intelligent Agents

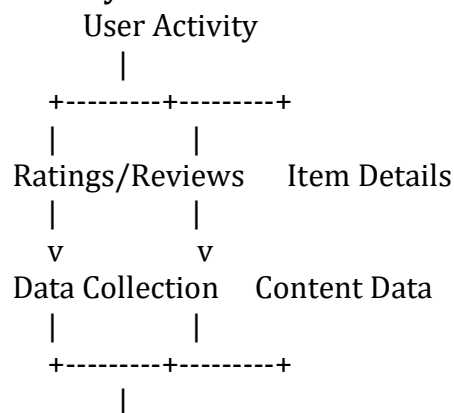
2. Problem Statement

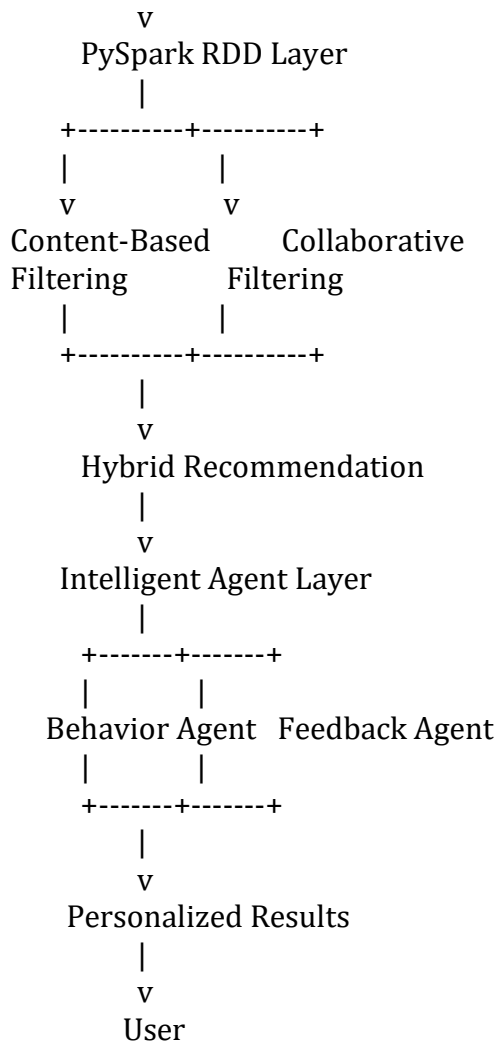
- Traditional recommendation systems may rely only on content similarity or user behavior.
- A hybrid system combines both approaches to produce more accurate and personalized recommendations.
- The system must process large numbers of users, items, ratings, clicks, searches, and feedback efficiently.

3. Objectives

- Process large-scale user-item data using PySpark RDDs.
- Implement content-based filtering.
- Implement collaborative filtering.
- Combine both recommendation scores.
- Use intelligent agents to monitor user behavior.
- Adapt recommendations based on real-time feedback.
- Provide scalable and fault-tolerant processing.

4. System Architecture





5. PySpark RDD Processing

Example input:

User Item Rating

U1 I1 5

U1 I2 4

U2 I1 4

U2 I3 5

U3 I2 5

Create an RDD:

```
ratings = sc.textFile("ratings.csv")
```

```
rating_rdd = ratings.map(
    lambda x: x.split(",")
)
```

Filter valid records:

```
valid_ratings = rating_rdd.filter(
    lambda x: len(x) == 3
)
```

Create user-item pairs:

```
user_items = valid_ratings.map(
    lambda x: (x[0], (x[1], float(x[2])))
)
```

Calculate average rating:

```
avg_rating = valid_ratings.map(
    lambda x: (x[1], float(x[2]))
).groupByKey().mapValues(
    lambda ratings: sum(ratings) / len(ratings)
)
```

Useful RDD operations to explain:

RDD Operation Purpose

map()	Transform records
filter()	Remove invalid/unwanted records
flatMap()	Generate multiple records
groupByKey()	Group data by user/item
reduceByKey()	Aggregate values efficiently
join()	Combine different datasets
sortBy()	Rank recommendations
collect()	Retrieve final results

6. Content-Based Filtering

The system analyzes item characteristics such as:

- Category
- Keywords
- Genre
- Description
- Features

For example:

User likes:

Machine Learning

Python

Artificial Intelligence

Recommended:

Deep Learning

Data Science

Neural Networks

The recommendation score can be represented as:

Content Score = Similarity(User Profile, Item Features)

7. Collaborative Filtering

The system analyzes similar users.

For example:

User A → Movie 1, Movie 2, Movie 3

User B → Movie 1, Movie 2

Since A and B have similar preferences,

Movie 3 can be recommended to User B.

A collaborative score can be represented as:

Collaborative Score = Similarity(User, Similar Users) × Their Ratings

8. Hybrid Recommendation

Combine both scores:

Hybrid Score = α (Content Score) + β (Collaborative Score)

For example:

Content Score = 0.80

Collaborative Score = 0.70

$\alpha = 0.6$

$\beta = 0.4$

Hybrid Score

$= (0.6 \times 0.80) + (0.4 \times 0.70)$

$= 0.48 + 0.28$

$= 0.76$

Therefore, the item receives a **hybrid recommendation score of 0.76**.

9. Intelligent Agents

Use multiple agents with different responsibilities:

1. User Behavior Agent

- Monitors clicks, searches, ratings, purchases, and watch time.
- Updates the user's preference profile.

2. Recommendation Agent

- Generates candidate items.
- Combines content and collaborative scores.
- Ranks recommendations.

3. Feedback Agent

- Monitors likes, dislikes, skips, and ratings.
- Determines whether recommendations were successful.

4. Adaptation Agent

- Changes recommendation weights based on user behavior.

Example:

User frequently clicks AI courses

↓

Behavior Agent detects pattern

↓

User profile updated

↓

Adaptation Agent increases AI preference

↓

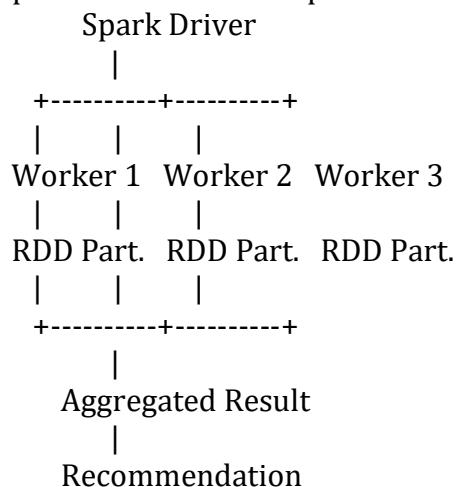
Recommendation Agent prioritizes AI courses

↓

Personalized recommendations

10. Distributed Processing

PySpark distributes RDD partitions across multiple worker nodes.



This allows millions of user-item interactions to be processed in parallel.

11. Scalability

The system can scale by:

- Increasing worker nodes.
- Partitioning RDDs.
- Using parallel transformations.
- Caching frequently accessed RDDs.
- Performing distributed aggregation.
- Processing new user interactions continuously.

12. Data Flow

User Interaction



Data Collection



RDD Creation



Data Cleaning



Feature Extraction



Content-Based Filtering



Collaborative Filtering



Score Combination



Intelligent Agent Analysis



Ranking



Personalized Recommendation



User Feedback



Model/Agent Adaptation

13. Innovation

A good innovative feature for your project is an **Adaptive Weighting Agent**.

Instead of keeping:

Content = 50%

Collaborative = 50%

the agent dynamically changes the weights.

For a **new user**:

Content = 80%

Collaborative = 20%

because there is little historical user data.

For an **experienced user**:

Content = 30%

Collaborative = 70%

because sufficient interaction history is available.

This directly demonstrates **AI + intelligent agents + personalization + real-time adaptation**.

14. Expected Output

User ID: U101

Recommended Items:

1. Artificial Intelligence Score: 0.92
2. Machine Learning Score: 0.88
3. Deep Learning Score: 0.84
4. Data Science Score: 0.79
5. Python Programming Score: 0.75

Agent Feedback:

User shows strong interest in AI and ML.

Recommendation weights:

Content-Based = 0.35

Collaborative = 0.65

System Status:

Personalization Updated Successfully

15. How this satisfies your rubric

Rubric	What you demonstrate
Problem Understanding – 10%	Recommendation-system problem, objectives and constraints
Architecture – 20%	Complete distributed architecture diagram
PySpark/RDD – 15%	map, filter, reduceByKey, groupByKey, join, sortBy
Intelligent Agents – 15%	Behavior, recommendation, feedback and adaptation agents
Scalability – 10%	RDD partitions, worker nodes, parallel processing
Data Flow – 10%	Complete input → processing → recommendation pipeline
Innovation – 10%	Adaptive recommendation weighting
Presentation – 10%	Diagrams, tables, pseudocode, sample output and explanation